



Архитектура системы Neuronline - Анализ профиля и генерация фитнес-плана

Tags	 Тренировки  Технические документы  Персонализированные планы
Owner	 BMD ONE
AI summary	- Анализ профиля пользователя и нейротестирование для генерации фитнес-плана. - Проверка доступа к генератору тренировок. - Генерация персонализированного плана тренировок и рекомендаций. - Мониторинг прогресса пользователя. - Структурированный JSON-ответ. - Структура маршрутов для приложения на Next.js с компонентами страниц. - Блок-схема процессов системы Neuronline.
AI translation	Система Neuronline включает сбор и анализ пользовательских данных, проверку доступа, генерацию персонализированных тренировочных планов, рекомендации и мониторинг прогресса, с использованием структурированного JSON-ответа для отслеживания достижений.

flowchart TD

%% Основные этапы

Start[["Начало"]] → InputData["Входные данные"]

%% Входные данные и анализ профиля

InputData → ["Данные пользователя"] | UserProfile["Профиль пользовате

InputData → |"Результаты тестов"| NeuroTest["Нейротестирование"]

%% Анализ профиля

UserProfile → Demographics["Демографические данные:

- Возраст
- Пол
- Вес/Рост
- Цели"]

NeuroTest → NeuroMetrics["Нейрометрики:

- Баланс
- Координация
- Реакция
- Память"]

%% Проверка доступа

Demographics → AccessCheck{"Проверка доступа"}

NeuroMetrics → AccessCheck

AccessCheck → |"Нет доступа"| Error["Ошибка доступа"]

AccessCheck → |"Есть доступ"| PlanGeneration["Генерация плана"]

%% Генерация плана

PlanGeneration → WorkoutPlan["План тренировок"]

PlanGeneration → Recommendations["Рекомендации"]

%% Компоненты плана

WorkoutPlan → Exercises["Упражнения:

- Нейрозарядка
- Основная тренировка
- Дополнительная"]

Recommendations → Support["Система поддержки:

- Советы
- Уведомления
- Мотивация"]

%% Формирование JSON

Exercises → JsonFormat["JSON-ответ"]

Support → JsonFormat

JsonFormat → Progress["Отслеживание прогресса:

- Выполненные тренировки
- Достижения
- Статистика"]

Progress → End[["Конец"]]

%% Стили

classDef process fill:#e1f5fe,stroke:#01579b,stroke-width:2px

classDef decision fill:#fff3e0,stroke:#ef6c00,stroke-width:2px

classDef data fill:#e8f5e9,stroke:#2e7d32,stroke-width:2px

class Start,End process

class AccessCheck decision

class InputData,Demographics,NeuroMetrics,WorkoutPlan,Recommendation

Блок-схема отображает основные компоненты и процессы системы Neuronline:

- Сбор и анализ входных данных (профиль пользователя и нейротестирование)
- Проверка доступа к генератору тренировок
- Генерация персонализированного плана тренировок
- Формирование рекомендаций и системы поддержки
- Создание структурированного JSON-ответа
- Мониторинг прогресса и достижений пользователя

Here's a complete route structure for the Next.js app router, implementing layouts for each page and focusing on page generation and navigation flow:

```
src/  
└─ app/  
    └─ layout.tsx
```

```

├── page.tsx
├── plan/
│   ├── layout.tsx
│   ├── page.tsx
│   ├── create/
│   │   └── page.tsx
│   └── [planId]/
│       ├── page.tsx
│       └── edit/
│           └── page.tsx
├── workouts/
│   ├── layout.tsx
│   ├── page.tsx
│   └── [workoutId]/
│       └── page.tsx
├── progress/
│   ├── layout.tsx
│   ├── page.tsx
│   └── [progressId]/
│       └── page.tsx
└── profile/
    ├── layout.tsx
    ├── page.tsx
    └── settings/
        └── page.tsx

```

Here's a breakdown of each route and its corresponding page component:

1. `app/layout.tsx`:

- The root layout component that wraps the entire application.
- Includes the main navigation and any global components.

2. `app/page.tsx`:

- The home page component.
- Displays the main content for the fitness workout app, including the quick starter, categories, and events sections.

3. `app/plan/layout.tsx`:

- The layout component for the workout plan pages.
- Includes any common components or navigation specific to the plan pages.

4. ``app/plan/page.tsx``:
 - The main workout plan page component.
 - Displays an overview of the user's workout plans.
5. ``app/plan/create/page.tsx``:
 - The page component for creating a new workout plan.
 - Includes a form or interactive components for creating a personalized workout plan.
6. ``app/plan/[planId]/page.tsx``:
 - The page component for viewing a specific workout plan.
 - Displays the details and exercises for a selected workout plan.
7. ``app/plan/[planId]/edit/page.tsx``:
 - The page component for editing a specific workout plan.
 - Allows the user to modify and update their workout plan.
8. ``app/workouts/layout.tsx``:
 - The layout component for the workout pages.
 - Includes any common components or navigation specific to the workout pages.
9. ``app/workouts/page.tsx``:
 - The main workouts page component.
 - Displays a list or grid of available workouts.
10. ``app/workouts/[workoutId]/page.tsx``:
 - The page component for viewing a specific workout.
 - Displays the details, instructions, and media for a selected workout.
11. ``app/progress/layout.tsx``:
 - The layout component for the progress tracking pages.
 - Includes any common components or navigation specific to the progress pages.
12. ``app/progress/page.tsx``:
 - The main progress tracking page component.
 - Displays an overview of the user's workout progress and achievements.
13. ``app/progress/[progressId]/page.tsx``:
 - The page component for viewing specific progress details.
 - Displays detailed information and analytics for a selected progress milestone.
14. ``app/profile/layout.tsx``:
 - The layout component for the user profile pages.
 - Includes any common components or navigation specific to the profile

pages.

15. ``app/profile/page.tsx``:

- The main user profile page component.
- Displays the user's profile information, preferences, and settings.

16. ``app/profile/settings/page.tsx``:

- The page component for managing user profile settings.
- Allows the user to update their profile information and preferences.

This route structure provides a clear separation of concerns and allows for easy navigation between different sections of the fitness workout app. Each route has its own layout component to handle common elements and a page component to render the specific content for that route.

Remember to implement the necessary components, state management, and data fetching logic within each page component to ensure a fully functional application.